

MYSQL PATCHING RUNBOOK - UBUNTU

=====

PURPOSE

This document describes a production-style procedure for patching MySQL within the SAME MySQL release series.

Example:

MySQL 8.4.6 -> MySQL 8.4.7

IMPORTANT:

This procedure is NOT intended for major/release-series upgrades such as:

MySQL 5.7 -> MySQL 8.0

MySQL 8.0 -> MySQL 8.4

Those upgrades require additional compatibility and upgrade checks.

Always test the procedure in DEV/TEST/UAT before Production.

=====

1. VARIABLES / ENVIRONMENT

=====

Set these according to your server.

BACKUP_DIR="/backup/mysql"

LOG_DIR="/var/log/mysql-patching"

Create directories:

```
sudo mkdir -p "$BACKUP_DIR"
```

```
sudo mkdir -p "$LOG_DIR"
```

Check:

```
df -h
```

```
df -i
```

=====

2. PRE-PATCHING CHECKS

=====

2.1 Check Ubuntu version

```
lsb_release -a
```

OR:

```
cat /etc/os-release
```

2.2 Check MySQL version

```
mysql --version
```

Login:

```
mysql -uroot -p
```

Run:

```
SELECT VERSION();
```

2.3 Check MySQL service

```
sudo systemctl status mysql --no-pager
```

Expected:

Active: active (running)

2.4 Check MySQL uptime

Login to MySQL:

```
mysql -uroot -p
```

Run:

```
SHOW GLOBAL STATUS LIKE 'Uptime';
```

2.5 Check MySQL important configuration

```
SHOW VARIABLES LIKE 'datadir';  
SHOW VARIABLES LIKE 'log_bin';  
SHOW VARIABLES LIKE 'binlog_format';  
SHOW VARIABLES LIKE 'server_id';  
SHOW VARIABLES LIKE 'port';  
SHOW VARIABLES LIKE 'max_connections';
```

2.6 Check MySQL configuration files

```
sudo mysql --help | grep -A 1 "Default options"
```

Common files:

```
/etc/mysql/my.cnf  
/etc/mysql/mysql.conf.d/mysqld.cnf
```

2.7 Backup MySQL configuration

```
sudo cp -a /etc/mysql /root/mysql-config-backup-$(date +%Y%m%d_%H%M%S)
```

Verify:

```
ls -lah /root/mysql-config-backup-*
```

=====

3. CHECK DISK SPACE

=====

Run:

`df -h`

`df -i`

Pay particular attention to:

`/var/lib/mysql`
`/backup`
`/`

DO NOT PATCH if the filesystem is almost full.

=====

4. CHECK MYSQL ERROR LOG BEFORE PATCHING

=====

Run:

`sudo journalctl -u mysql --since "2 hours ago"`

Also check:

`sudo tail -100 /var/log/mysql/error.log`

Look for serious errors such as:

ERROR
InnoDB
corruption
crash
failed
permission denied

Resolve existing serious problems before patching.

=====

5. CHECK REPLICATION

=====

IF THIS SERVER IS NOT A REPLICA, SKIP THIS SECTION.

For MySQL 8:

`mysql -uroot -p`

`SHOW REPLICA STATUS\G`

Check:

Replica_IO_Running: Yes
Replica_SQL_Running: Yes

Also check replication lag:

Seconds_Behind_Source: 0

For older MySQL terminology:

SHOW SLAVE STATUS\G

If replication is already broken, DO NOT PATCH until the problem is understood and resolved.

6. CHECK BINARY LOG

Login:

```
mysql -uroot -p
```

Check:

```
SHOW VARIABLES LIKE 'log_bin';
```

If binary logging is enabled:

```
SHOW BINARY LOG STATUS;
```

For older versions:

```
SHOW MASTER STATUS;
```

Also record GTID if enabled:

```
SELECT @@GLOBAL.gtid_executed;
```

SAVE THIS INFORMATION.

7. CHECK AVAILABLE MYSQL PATCH

Update APT:

```
sudo apt update
```

Check available MySQL updates:

```
apt list --upgradable 2>/dev/null | grep -i mysql
```

Also:

```
apt-cache policy mysql-server
```

Example:

Installed: 8.4.6

Candidate: 8.4.7

GOOD:

8.4.6 -> 8.4.7

STOP AND INVESTIGATE:

8.0.x -> 8.4.x

5.7.x -> 8.0.x

Those are release-series/major upgrades and require a different procedure.

8. SIMULATE THE PATCH

Before installing anything:

```
sudo apt-get --simulate install mysql-server
```

Review the output carefully.

Check that:

- MySQL packages are being updated
- No unexpected packages are removed
- No unexpected major version change is proposed

If unexpected packages are being removed, STOP.

9. TAKE FULL XTRABACKUP

IMPORTANT:

Do this before patching production.

Check XtraBackup:

```
sudo xtrabackup --version
```

Create backup directory:

```
sudo mkdir -p /backup/mysql
```

Set timestamp:

```
BACKUP_NAME="prepatch_$(date +%Y%m%d_%H%M%S)"
```

Run FULL backup:

```
sudo xtrabackup \  
  --backup \  
  --target-dir="/backup/mysql/$BACKUP_NAME"
```

Wait for:

completed OK!

Check backup:

```
sudo ls -lah /backup/mysql/
```

```
sudo du -sh "/backup/mysql/$BACKUP_NAME"
```

IMPORTANT:

The backup should ideally be stored on a separate disk/server.

Do not rely only on a backup stored on the same disk as
/var/lib/mysql.

10. VERIFY XTRABACKUP

Check:

```
sudo ls -lah /backup/mysql/
```

```
sudo du -sh /backup/mysql/*
```

Look at the backup output/log.

You want:

completed OK!

For production environments, periodically perform a TEST RESTORE
of your XtraBackup backups.

A backup that has never been restored should not be considered
fully verified.

11. RECORD CURRENT MYSQL INFORMATION

Before patching, record:

```
mysql -uroot -p
```

Run:

```
SELECT VERSION();
```

```
SHOW VARIABLES LIKE 'datadir';
```

```
SHOW VARIABLES LIKE 'log_bin';
```

```
SHOW VARIABLES LIKE 'binlog_format';
```

SHOW BINARY LOG STATUS;

If GTID is enabled:

SELECT @@GLOBAL.gtid_executed;

Save the output.

12. CHECK APPLICATION STATUS

Before patching production:

- Notify application team
- Schedule maintenance window
- Stop/drain application traffic
- Make sure no important database transactions are running

Expected flow:

```
Application
|
v
Drain/Stop
|
v
MySQL patch
|
v
MySQL verification
|
v
Application start
|
v
Application testing
```

13. FINAL CHECK BEFORE PATCH

Run:

mysqladmin -uroot -p ping

Expected:

mysqld is alive

Check:

sudo systemctl status mysql --no-pager

Check disk:

df -h

Check backup:

```
sudo ls -lah /backup/mysql/
```

Check current version:

```
mysql --version
```

If everything is OK, proceed.

```
=====
```

14. APPLY MYSQL PATCH

```
=====
```

For a normal patch within the same release series:

```
sudo apt-get install mysql-server
```

DO NOT blindly run:

```
sudo apt upgrade -y
```

on production without first checking exactly what packages will be changed.

```
=====
```

15. MONITOR MYSQL DURING PATCH

```
=====
```

Open another SSH session and run:

```
sudo journalctl -fu mysql
```

In another terminal:

```
sudo systemctl status mysql
```

If MySQL does not automatically start:

```
sudo systemctl start mysql
```

Then:

```
sudo systemctl status mysql --no-pager
```

```
=====
```

16. VERIFY MYSQL VERSION AFTER PATCH

```
=====
```

Run:

```
mysql --version
```

Then:

```
mysql -uroot -p
```



```
SELECT VERSION();
```

Example:

BEFORE: 8.4.6

AFTER: 8.4.7

```
=====
17. CHECK MYSQL SERVICE
=====
```

Run:

```
sudo systemctl status mysql --no-pager
```

Expected:

Active: active (running)

Also:

```
mysqladmin -uroot -p ping
```

Expected:

mysql is alive

```
=====
18. CHECK MYSQL ERROR LOG AFTER PATCH
=====
```

Run:

```
sudo journalctl -u mysql --since "30 minutes ago"
```

Also:

```
sudo tail -100 /var/log/mysql/error.log
```

Look for:

```
ERROR
InnoDB errors
crash
failed
corruption
permission problems
```

Investigate unexpected errors.

```
=====
19. CHECK DATABASES
=====
```

Login:

```
mysql -uroot -p
```

Run:

```
SHOW DATABASES;
```

Select application database:

```
USE myapp;
```

Check tables:

```
SHOW TABLES;
```

Run a basic query:

```
SELECT COUNT(*) FROM users;
```

Test important tables and queries used by the application.

```
=====
20. CHECK BINARY LOGGING AFTER PATCH
=====
```

Run:

```
SHOW VARIABLES LIKE 'log_bin';
```

If enabled:

```
SHOW BINARY LOG STATUS;
```

Verify that binary logging is still enabled if your backup, PITR, or replication design depends on it.

```
=====
21. CHECK REPLICATION AFTER PATCH
=====
```

IF THIS SERVER IS A REPLICA:

```
SHOW REPLICA STATUS\G
```

Verify:

```
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
```

Check:

```
Seconds_Behind_Source
```

If replication does not recover, investigate before allowing normal production traffic.

=====

22. APPLICATION TESTING

=====

Test:

1. Application login
2. Database connection
3. SELECT operation
4. INSERT operation
5. UPDATE operation
6. Application transaction
7. Important reports
8. Application logs

The patch is NOT considered successful only because MySQL service is "active".

The application must also work.

=====

23. POST-PATCH MONITORING

=====

Monitor:

CPU
RAM
Disk
Disk I/O
MySQL connections
Query latency
Slow queries
MySQL errors
Replication lag
Application errors

Basic Linux commands:

top

free -h

df -h

MySQL:

SHOW GLOBAL STATUS LIKE 'Threads_connected';

SHOW GLOBAL STATUS LIKE 'Threads_running';

SHOW GLOBAL STATUS LIKE 'Slow_queries';

Monitor for several hours after patching.

24. ROLLBACK PLAN

IMPORTANT:

DO NOT immediately delete:

`/var/lib/mysql`

DO NOT immediately overwrite production data.

If MySQL fails after patching:

1. Check service status

`sudo systemctl status mysql`

2. Check logs

`sudo journalctl -u mysql -n 200 --no-pager`

3. Check MySQL error log

`sudo tail -200 /var/log/mysql/error.log`

4. Determine the cause

5. If database recovery is required, use the verified XtraBackup backup.

25. PREPARE XTRABACKUP FOR RECOVERY

Example:

```
xtrabackup \  
  --prepare \  
  --target-dir=/backup/mysql/prepatch_YYYYMMDD_HHMMSS
```

IMPORTANT:

Always verify your XtraBackup version is compatible with your MySQL version before performing recovery.

For incremental backups, prepare the FULL backup first and then apply each incremental backup in order.

=====

26. RECOVERY DESIGN

=====

Recommended recovery process:

```
graph TD
    A[XtraBackup FULL] --> B[Prepare backup]
    B --> C[Restore to separate MySQL server]
    C --> D[Apply binary logs if required]
    D --> E[Verify data]
    E --> F[Application testing]
    F --> G[Switch application to recovered server]
```

Do NOT experiment with the only copy of production data.

=====

27. MYSQL PATCHING SCRIPT

=====

The following script performs basic pre-checks and starts the XtraBackup process.

IMPORTANT:

Review and customize this script before production use.

Create:

```
sudo nano /usr/local/sbin/mysql-prepatch.sh
```

Paste:

```
#!/bin/bash
```

```
set -e
```

```
BACKUP_ROOT="/backup/mysql"
```

```
LOG_DIR="/var/log/mysql-patching"
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```
BACKUP_DIR="${BACKUP_ROOT}/prepatch_${DATE}"
```

```
LOG_FILE="${LOG_DIR}/prepatch_${DATE}.log"
```

```
mkdir -p "$BACKUP_ROOT"
mkdir -p "$LOG_DIR"

exec >>(tee -a "$LOG_FILE") 2>&1

echo "====="
echo "MYSQL PRE-PATCH CHECK"
echo "====="
echo "Date: $(date)"
echo

echo "1. Ubuntu version"
lsb_release -a || true
echo

echo "2. MySQL version"
mysql --version
echo

echo "3. MySQL service"
systemctl is-active mysql
echo

echo "4. Disk space"
df -h
echo

echo "5. Inode usage"
df -i
echo

echo "6. XtraBackup version"
xtrabackup --version
echo

echo "7. Creating XtraBackup directory"
mkdir -p "$BACKUP_DIR"

echo
echo "8. Starting FULL XtraBackup"
echo "Backup directory: $BACKUP_DIR"
echo

xtrabackup \
  --backup \
  --target-dir="$BACKUP_DIR"

echo
echo "====="
echo "BACKUP COMPLETED"
echo "====="

echo "Backup location:"
echo "$BACKUP_DIR"

echo
```

```
echo "Backup size:"
du -sh "$BACKUP_DIR"
```

```
echo
echo "Completed at:"
date
```

```
echo
echo "IMPORTANT:"
echo "Do NOT continue automatically with patching."
echo "Verify the backup before applying the MySQL patch."
```

```
=====
28. MAKE SCRIPT EXECUTABLE
=====
```

Run:

```
sudo chmod 750 /usr/local/sbin/mysql-prepatch.sh
```

```
=====
29. RUN PRE-PATCH SCRIPT
=====
```

Run:

```
sudo /usr/local/sbin/mysql-prepatch.sh
```

After it completes:

```
sudo ls -lah /backup/mysql/
```

Find your latest backup:

```
sudo ls -lht /backup/mysql/
```

Verify the backup was completed successfully.

```
=====
30. PATCH COMMAND
=====
```

After manually reviewing and approving the backup:

```
sudo apt update
```

Check:

```
apt list --upgradable 2>/dev/null | grep -i mysql
```

Simulate:

```
sudo apt-get --simulate install mysql-server
```

If everything is correct:

```
sudo apt-get install mysql-server
```

```
=====
```

31. POST-PATCH COMMANDS

```
=====
```

Run:

```
mysql --version
```

```
sudo systemctl status mysql --no-pager
```

```
mysqladmin -uroot -p ping
```

Login:

```
mysql -uroot -p
```

Run:

```
SELECT VERSION();
```

```
SHOW DATABASES;
```

```
SHOW VARIABLES LIKE 'log_bin';
```

```
SHOW BINARY LOG STATUS;
```

If replica:

```
SHOW REPLICA STATUS\G
```

Check error log:

```
sudo journalctl -u mysql --since "30 minutes ago"
```

```
=====
```

32. SUCCESS CRITERIA

```
=====
```

The patch is considered successful when ALL of the following are true:

[] MySQL service is running

- [] New MySQL version is confirmed
- [] mysqladmin ping succeeds
- [] No serious errors in MySQL error log
- [] All required databases are available
- [] Important tables are accessible
- [] Binary logging is working if required
- [] Replication is healthy if applicable
- [] Application can connect to MySQL
- [] Application SELECT operations work
- [] Application INSERT/UPDATE operations work
- [] Application has no database errors
- [] CPU/RAM/disk usage is normal
- [] Backup is safely retained

=====

33. IMPORTANT PRODUCTION RULES

=====

1. Always take a verified backup before patching.
2. Keep the backup outside the MySQL data directory.
3. Ideally keep another copy on a separate server/storage system.
4. Test backups by performing regular restore tests.
5. Never patch production for the first time.
6. Test the same patch on DEV/TEST/UAT first.
7. Check replication before and after patching.
8. Check binary logging before and after patching.
9. Do not blindly use "apt upgrade -y" for production MySQL.
10. Never delete /var/lib/mysql as a first troubleshooting step.
11. Never restore a backup directly over production without understanding the recovery procedure.
12. Keep a maintenance window.
13. Monitor the application after patching.

14. Keep the pre-patch backup until the patch has been successfully validated.

15. Document the old and new MySQL versions.

=====

34. CHANGE RECORD

=====

Server Name:

Environment:

DEV / TEST / UAT / PROD

Ubuntu Version:

MySQL Version BEFORE:

MySQL Version AFTER:

XtraBackup Version:

Patch Date:

Patch Start Time:

Patch End Time:

Backup Location:

Backup Size:

Replication:

YES / NO

Replication Status Before:

Replication Status After:

Application Status:

Issues:

Performed By:

Approved By:

=====
END OF MYSQL PATCHING RUNBOOK
=====